Forecasting IPC Code Popularity Using Graph Neural Networks and Text Embeddings

Samuel Rauh

20.03.2026

1 Introduction

The goal of this project is to predict the popularity of different technologies by forecasting how often specific International Patent Classification (IPC) codes will appear in patents over the following year. IPC codes are a global system that is used to categorize patents into specific technical fields. For example, all codes starting with "A" are for human necessities while all codes starting with "H" are for electronics [9]. The codes are then further hierarchically classified into a total of around 70,000. Having the ability to predict the future popularity of IPC codes can have many benefits for companies, investors, and governments. The gained insight can lead to more stable supply chains, better trade deals, and more strategic R&D investments, as it helps them anticipate where the market is moving. This project is based on the work of Aroyehun et al. [2]. With the help of LLMs, they created text embeddings for many IPC codes. That is, each IPC code is placed in a multidimensional space, based on what patents it is used in and how these patents are described. We can now use these embeddings for our project with the idea that the placement and movement of an IPC code in the embedding space can somehow be an indicator of its future popularity. To explore this, I am training three different models: a standard Multilayer Perceptron (MLP) just using the embeddings and a couple of specific scalar values, a Graph Neural Network (GNN) that maps the relationships between codes based on co-occurrences, and a Temporal GNN that combines a GNN with a Gated Recurrent Unit (GRU) to track how those technological relationships evolve over time.

2 Data

The data from Aroyehun et al. [2] contains an embedding for each patent based on its abstract description. Patents with a similar description will be placed close to each other in the multidimensional space, while patents different from one another are further away. The specifically fine-tuned models that created the final embeddings produce a vector with 1024 dimensions for each patent.

Since we are interested in the prediction of future popularity, we split the data by year. The data only provides the embeddings for the patents themselves, so we first have to calculate the embedding of the IPC codes. This is done by calcuation the mean of all the patents an IPC code is used in during the respective year. In the end, this gives us a dataset of IPC code embeddings for each year from 2006 to 2023.

As there is not a lot of movement when looking at the total popularity of patents, we have created a target score that is calculated based on the relative change in popularity.

The distribution of occurrences of IPC codes is very skewed, with many codes only appearing in a small number of patents and a small number of very prominent codes appearing in many patents. This is also something we need to account for when calculating our score.

As a first step, we remove all the IPC codes with fewer than 20 occurrences in a year, since for such low numbers, a slight increase would already boost a relative gain in occurrences by a lot while still not being very important in the greater scheme of things. From there, we calculate the target score as the relative count difference.

$$\text{Relative Count Difference} = \frac{\text{count}_{\text{next}} - \text{count}}{\text{count}}$$

This score gives us a good measure for how much a technology code's popularity changes from one year to the next, while accounting for the current popularity of the code and still keeping a balanced distribution without many outliers, as shown in Figure 1.

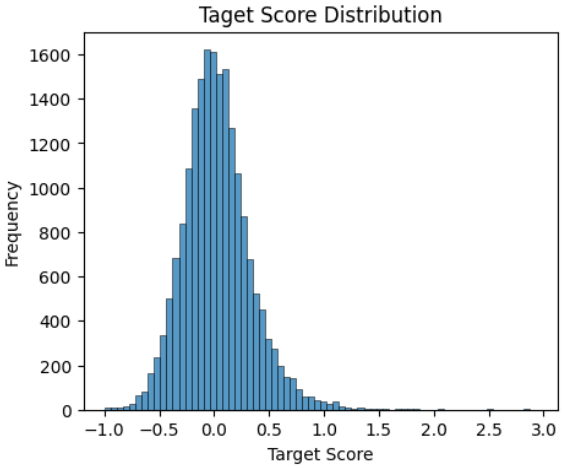

Figure 1: Target Variable Distribution.

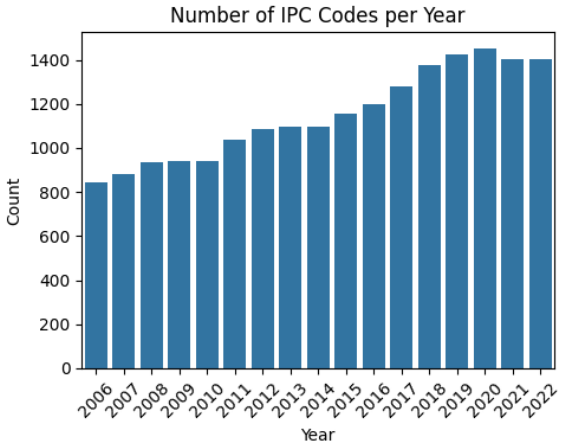

Figure 2: Number of IPC codes per year.

During the project, we also experimented with different calculations of the target score, such as the logarithmic count difference and also more complex calculations involving the inverse sine. The results were similar for all of the scores, which is why we chose the relative count difference as a simpler and more interpretable option.

Figure 2 shows the number of data points in each year after the final cleaning and score calculation. In total, there are 19,542 data points over 17 years. The year 2023 is removed since we need a target score from the following year.

In addition to the embedding data, we added scalar features containing the standardized counts of the current and the previous years, as well as the calculated score of the current and the previous year, and the main category of the IPC code.

To construct the graph structure for the Graph Neural Networks (GNN), we relied on co-occurrences of IPC codes in patents. To calculate the edge weights, we use Salton's Cosine Similarity, which weights the number of co-occurrences of two IPC codes by both of their counts. As edge features, we also added the standardized count, as well as both of these measures from the previous year, and the change in both measures from the previous to the current year, to try to capture the temporal dynamic even for the non-temporal, static GNN.

The data is then finally split into training (2006 - 2017), validation (2018) and test set (2019 - 2022).

3 Methodology

To predict the future popularity of IPC codes, we test three different kinds of Neural Networks (NN). We are using Python as the programming language and PyTorch as the framework for the neural networks, as well as Optuna for the hyperparameter tuning [7, 1].

The first model is a simple Multilayer Perceptron (MLP) with seven fully connected layers. As inputs, it takes the embedding of the IPC code as well as the scalar features which are concatenated. The inputs are then simply passed forward through all the layers using linear layers, a ReLU activation function, and dropout at each layer.

The second model is a Graph Neural Network (GNN). For the creation of the GNNs, we rely on PyTorch Geometric, a library built on top of PyTorch to handle graph data and train GNNs [5]. To process the graph structure, we use the GATv2 layers [3]. These layers use

a dynamic graph attention mechanism which determines how much each node influences the other nodes based on the node and edge attributes. This creates a ranking for each individual node on how it is influenced by its neighbors. This also enables the usage of multiple edge attributes, which is not possible in many other graph layer types. These GNN layers create a graph embedding for each node in the network, which encodes the original node data but enriches it with the information from the graph structure. Finally, the embeddings are then passed through a sequence of fully connected layers using ReLU and dropout to predict our performance score.

The third and final model is a combination of a GNN with a recurrent neural network with the goal of incorporating the temporal aspect of this task. The data is first passed through layers of a graph attention network, the same ones used for the GNN. The graph embeddings then additionally pass through layers of a gated recurrent unit (GRU) [4]. The GRU is designed to handle sequential or temporal data. It acts like a memory. With "gates", it decides what information from the current step should be remembered for the next step and what old memory should be forgotten. The memory state from the previous years is then passed into the model together with the graph embedding from the current year to predict the popularity score of the next year. Finally, the data is passed through a set of fully connected layers similar to the ones used in the previous models. The exact number of GAT and GRU layers is determined by hyperparameter tuning.

In addition to the proposed regression task, we also tried the classification of the top quartile IPC codes for the next year using Margin Ranking Loss, but the achieved results were not more successful than for the direct prediction of the score which is why we stuck with the more general regression task.

Finally, since our calculated score and the derived evaluation are not directly interpretable, we established two baselines to better assess the actual performance of our models. The first baseline uses the previous year's score as the direct predictor, and the second uses an overall average score.

4 Training

All three models were trained using a similar setup. As the loss function, we used the mean absolute error (L1 Loss) and the Adam optimizer. Before fitting the models on the full training data, we performed an overfitting test by just using one year to see if the models can converge and work as expected [6]. To define the hyperparameters of the models and training setup, we performed hyperparameter tuning using Optuna and the Tree-structured Parzen Estimator as a sampler [8]. The parameter tuning ran for 30 iterations each. Table 1 shows the trained parameters for each model, as well as the ones that yielded the best results during tuning.

After the hyperparameter tuning, the final models were trained for a maximum of 500 epochs using an early stopper with a patience of 50, meaning that the training is stopped once the loss does not improve for 50 consecutive epochs. The model state with the lowest validation loss is then restored and saved for evaluation.

Table 1 also shows the validation loss for each of the models after training them with the hyperparameters that performed the best. The MLP and the Temporal GNN performed similarly with a mean absolute error (MAE) of around 0.21. The GNN performed a little bit worse with an MAE of 0.227.

Figure 3 shows the learning curves of the final training of the three models, comparing the validation loss to the training loss. We can see that in all of the training runs, the best validation loss is achieved after only a relatively small number of epochs. The plots also show how the training loss decreases much more rapidly than the validation loss. We can also see that our training runs do not follow the anticipated steady decrease in loss one would expect from the successful training of neural networks.

Table 1: Best Hyperparameters and Validation Performance Identified via Optuna

Model	Hyperparameter	Optimal Value	Best Val. Loss
MLP	Weight Decay	4.151×10^{-5}	0.211
	Learning Rate	1.460×10^{-5}	
	Batch Size	104	
	Dropout	0.30	
GNN	Hidden Channels	16	0.227
	Out Channels	512	
	Heads	3	
	Dropout	0.30	
	Learning Rate	4.202×10^{-5}	
	Weight Decay	3.632×10^{-5}	
Temporal GNN	Hidden Channels	448	0.212
	GAT Heads	5	
	Num GAT Layers	3	
	Num GRU Layers	1	
	Dropout	0.40	
	Learning Rate	4.623×10^{-4}	
	Weight Decay	4.335×10^{-6}	

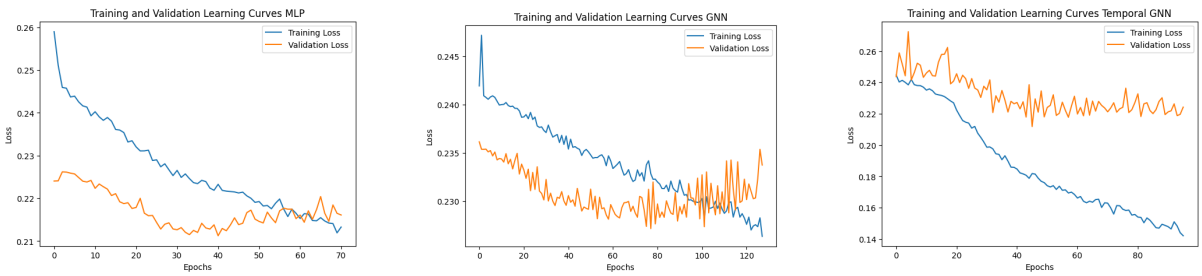

Figure 3: Learning Curves

5 Results

After fitting the models with their best-performing parameters, we evaluate them on the test set.

Table 2: Model Performance Comparison

Metric	MLP	GNN	Temporal GNN
MAE (Mean Absolute Error)	0.2217	0.2308	**0.1886**
RMSE (Root Mean Squared Error)	0.2921	0.3002	**0.2520**
MSE (Mean Squared Error)	0.0853	0.0901	**0.0635**
R^2 Score	0.0217	-0.0330	**0.2718**

Table 2 shows the performance of the three models on the test set using multiple evaluation metrics. We can see that the Temporal GNN performs best in all of the metrics by a rather large gap. It performs even better on the training set than on the validation set with an MAE of 0.189. The MLP and the GNN both perform a little worse than on the validation set with an MAE of 0.222 (MLP) and 0.231 (GNN).

Figures 4 and 6 show the predicted values against the actual values in a scatter plot, with the diagonal representing the perfect prediction. For the MLP and the GNN, we can again see similar tendencies. Both models seem to have an upper barrier for their prediction at around

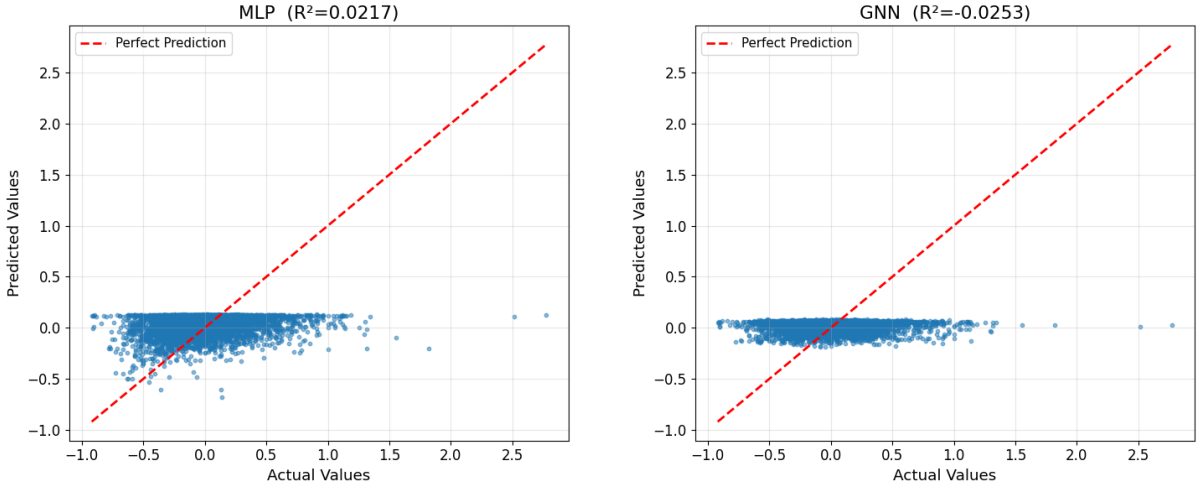

Figure 4: Predicted vs. Actual values for MLP and GNN

0.15, with many values lying on this line or below it. For the MLP, there is a slight tendency to the lower left half, which also reflects the small improvement in performance compared to the GNN. In contrast, the Temporal GNN clearly shows a tendency to follow the diagonal, reflecting its superior performance.

Table 3: Model Improvement Over Overall Mean Baseline

Model	**MAE Imp.**	**RMSE Imp.**	**Significant?** ($\alpha = 0.05$)	**p-value**
MLP	+0.64%	+1.09%	No	0.1400
GNN	-1.86%	-1.08%	No	1.0000
Temporal GNN	**+16.75%**	**+15.14%**	**Yes**	$<$ **0.0001**

Note: Positive percentages indicate improvement (error reduction) over the baseline.

When comparing our models to the established baselines, the overall average performed much better than using the previous score, which is why we focus on the average as our main comparison. In Table 3, we can see the models' performances against the mean baseline. Both the MLP and the GNN fail to significantly outperform the baseline, with the GNN even performing worse. The Temporal GNN improves a lot on the baseline with a 16.75 % MAE and a 15.14 % RMSE improvement and a very high significance. Figure 5 shows the results of the Temporal GNN compared to the two baselines. The statistical significance of the performance gain using the Temporal GNN is also shown in Figure 7. We performed a bootstrapping test using 1000 samples (with replacement) to test our evaluation for robustness. As we can see, the bootstrapped results for the Temporal GNN all performed better than the bootstrapped results for the mean predictor model, which gives us confidence that our model's improvement on the baseline is not just a coincidence.

6 Discussion

As mentioned before, the Temporal GNN's performance on the test set is better than on the validation set during training. Since we are training a model containing a GRU, the model needs multiple epochs to "warm up" to fill the hidden state of the GRU with useful information. The validation during our training is performed without such a warm up, which leads to worse performance. Also, the performance of such a temporal model can get better over time, since there is more information "saved" in the GRU from many previous years.

One notable anomaly during testing was the apparent prediction barrier at around 0.15 for both the MLP and the GNN. While we could not find a definitive explanation for this

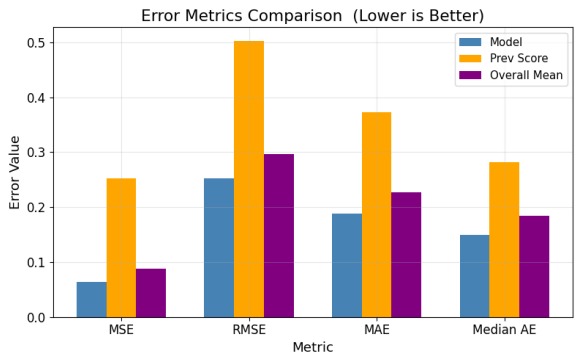

Figure 5: Temporal GNN Evaluation

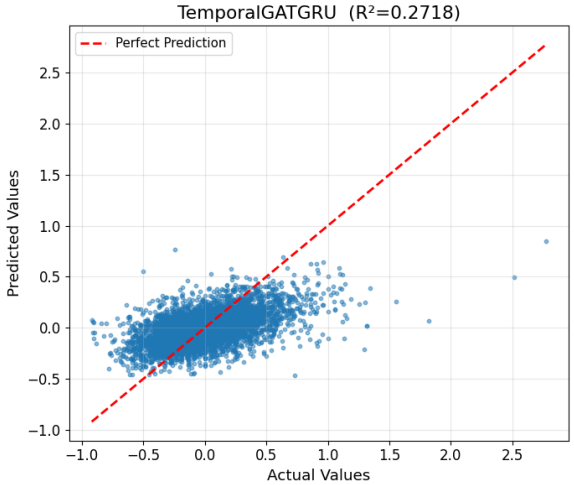

Figure 6: Predicted vs. Actual GNN

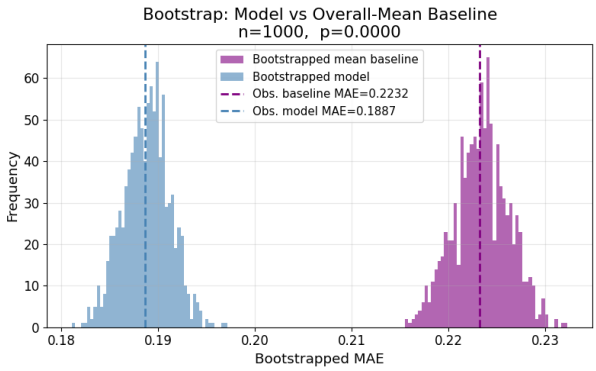

Figure 7: Temporal GNN Bootstrapped Evaluation vs. Baseline

behavior, we can guess that it may be tied to how the relative count difference is calculated, or it could indicate that high target score values are inherently unpredictable for these models. The Temporal GNN's ability to easily surpass this barrier supports the idea that historical context is required to predict large gains in popularity.

It was expected that the Temporal GNN would outperform the other two models, since it is a more complex model and is designed to work with time-series data. However, it was unexpected that the GNN performed so poorly. It is especially interesting that it performed worse than the mean baseline and the MLP, despite its more complex structure and the success of the Temporal GNN. This leads us to believe that the architecture of the model might have been flawed from the beginning despite

7 Conclusion

Above all, this project shows the power of pairing Graph Neural Networks with a sequential model like the Gated Recurrent Unit. By combining graph embeddings over the years, we created a model that consistently and significantly outperforms the benchmark. While there is still room for improvement, these results clearly highlight the potential of this methodology.

To further improve the models, there is still much room to try different methods in different steps of the process. Firstly, we could use more years of data to boost performance. Secondly, we could test even more neural network architectures. While we already compared multiple methods, there are still many options out there for both the GNN and the recurrent components. Thirdly, we could build different graph structures by adding edges based on extra variables—for example, if two IPC codes share the same category, or based on the Euclidean distance of the

vector embeddings. Finally, since our results depend on the calculated performance score, trying out a different formula might lead to even better outcomes. In conclusion, this project has provided a solid foundation for future research in this area.

References

[1] Takuya Akiba et al. "Optuna: A next-generation hyperparameter optimization framework". In: *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 2019, pp. 2623–2631.

[2] Segun Taofeek Aroyehun et al. "Anticipating Innovation: An LLM based approach". Unpublished manuscript. 2024.

[3] Shaked Brody, Uri Alon, and Eran Yahav. "How attentive are graph attention networks?" In: *arXiv preprint arXiv:2105.14491* (2021).

[4] Kyunghyun Cho et al. "On the properties of neural machine translation: Encoder–decoder approaches". In: *Proceedings of SSST-8, eighth workshop on syntax, semantics and structure in statistical translation*. 2014, pp. 103–111.

[5] Matthias Fey and Jan Eric Lenssen. "Fast graph representation learning with PyTorch Geometric". In: *arXiv preprint arXiv:1903.02428* (2019).

[6] Andrej Karpathy. *A Recipe for Training Neural Networks*. 2019. URL: `https://karpathy.github.io/2019/04/25/recipe/`.

[7] Adam Paszke et al. "Pytorch: An imperative style, high-performance deep learning library". In: *Advances in neural information processing systems* 32 (2019).

[8] Shuhei Watanabe. "Tree-structured parzen estimator: Understanding its algorithm components and their roles for better empirical performance". In: *arXiv preprint arXiv:2304.11127* (2023).

[9] World Intellectual Property Organization. *International Patent Classification (IPC)*. Accessed: 2026-03-12. WIPO. 2026. URL: `https://www.wipo.int/en/web/classification-ipc`.

Appendix

GitHub Repository: `https://github.com/samrauh/ds-project-2025-pub`